

Chai-Square Member Playbooks

Sep 25, 2026 · @Adithya Sankar Menon

How to use this doc

This doc is for Bhanu, Siva and Arushi. Each playbook lists your tasks in order, what counts as done, and ready-made prompts for an AI coding assistant (Claude Code, Cursor, Copilot Chat or ChatGPT). Adithya handles the GPU and training work separately. Task IDs (BH-1, SI-3, AR-5 ...) match `TEAM_PLAN.md` and the `NotImplementedError(...)` markers in the stubs.

Working with the AI assistant

1. Open the assistant with the repo root as its working folder, so it can read `TEAM_PLAN.md`, `docs/INTERFACES.md` and the stubs.
2. Paste the **session starter** below once at the start of every new chat. Then paste your task prompts one at a time, in order.
3. After each prompt, run what it wrote on real data and check the numbers against the task's done criteria before moving on. The assistant can't see the dataset unless it runs code, so ask it to run and print results.
4. If the assistant wants to change a column name, a file name or someone else's file, stop. Those are shared contracts; raise it in the team chat first.
5. Commit on your own branch after each task that works, then open a PR.

Session starter (paste first, every chat)

You are helping team Chai-Square in the Amazon ML Challenge 2026 (business entity resolution).

Read these files before writing any code: TEAM_PLAN.md, docs/INTERFACES.md, src/config.py, src/io_utils.py, and the stub file(s) for my task.

Task summary: for every Source 1 business record, find the Source 2 / Source 3 records that are the same real business. Scoring is macro F0.5 per Source 1 entity, so a wrong match costs more than a missed one, and an entity with no true matches scores 1.0 only if we predict nothing.

Hard rules:

- Read the raw TSVs only through src/io_utils.py (read_source, read_all_records, read_ground_truth).
- Follow docs/INTERFACES.md exactly for every file you read or write: file names, column names, dtypes. Records are keyed by (split, entity_id); pairs by (s1_id, cand_id).
- Country is an open set of strings (the test set has France, which train lacks). Never hard-code, filter or one-hot countries.
- No external data, APIs, geocoding or web lookups. No GPL libraries (e.g. no unicode). Stdlib, numpy, pandas, scipy, scikit-learn, rapidfuzz, jellyfish, pyarrow are fine.
- Windows machine: use `python`, and put multiprocessing entry points under `if \_\_name\_\_ == "\_\_main\_\_":`.
- Only edit the files I own. Keep code small, commented where it is not obvious, with a CLI entry point matching the stub.
- Respect config.SAMPLE_FRAC via io_utils.in_dev_sample() when I develop on a sample.

My name and files: <PASTE YOUR ROW FROM THE ROLES TABLE IN TEAM_PLAN.md>
Confirm you have read the files, then wait for my first task.

Shared setup (before the 10:00 kickoff)

Everyone finishes these five steps before 10:00 IST on 25 Sep, so the first handoffs at 12:00 can land.

- ☐ Accept the GitHub invite from Adithya, then clone: `git clone https://github.com/AdithyaSM31/Business-Entity-Resolution---Amazon-ML-Challenge-2026.git`
- ☐ Copy the challenge dataset/ folder (with train/ and test/) into the repo root. It is gitignored; never commit it.

- ❑ Copy `utils/validate_submission.py` and `Documentation_template.md` from `student_resource/` into the repo's `utils/` and `docs/` folders (Arushi commits them once).
- ❑ Python 3.11, then `pip install -r requirements.txt`. The CPU build of torch is fine for you.
- ❑ Check the readers work: `python -c "from src import io_utils as io; print(io.read_all_records().groupby(['split', 'source']).size())"`

If your laptop is slow, set `BER_SAMPLE_FRAC=0.2` (PowerShell:

`$env:BER_SAMPLE_FRAC="0.2"`) while developing. Anything you hand off or log must be run with the full data (unset it), or ask Adithya to run it.

Branch names: `bhanu/<topic>`, `siva/<topic>`, `arushi/<topic>`. Share cache files larger than a few MB (parquet, npy) through the team drive, not git.

Bhanu: data checks, normalization, pair features

You turn raw names and addresses into clean, comparable fields, then into the similarity features the model learns from. Your first two hours unblock everyone: Siva needs `records.parquet` v0 and Arushi needs your data-check answers by 12:00 on 25 Sep.

Files you own: `src/eda.py`, `src/normalize.py`, `src/features_pair.py`, `docs/EDA.md`.

Task	Due (IST)	Done when
BH-1 Data checks	25 Sep 12:00	<code>docs/EDA.md</code> answers (a)–(h) with numbers; (a)–(c) posted in chat
BH-2 Normalization v0	25 Sep 12:00	<code>cache/records.parquet</code> has every record of all 6 files; row count equals the sum of the file sizes
BH-3 Normalization v1	25 Sep 17:00	All INTERFACES columns filled; 30 random records per country eyeballed and look right
BH-4 Pair features v1	25 Sep 21:00	<code>feat_pair_train/test.parquet</code> has one row per candidate, about 40 <code>fn_ / fa_</code> columns, runs in under 15 min on full data
BH-5 Abbreviations, phonetics, French rules	26 Sep 13:00	Mined abbreviation table saved; France test records normalize sensibly
BH-6 Pair features v2	26 Sep 17:00	CV F0.5 from Adithya's next run does not drop; new features show up in importance

Task	Due (IST)	Done when
BH-7 Error-sheet fixes	26 Sep, after each run	Top name/address error patterns fixed or noted as unfixable
BH-8 Freeze	27 Sep 14:00	No changes to normalization or features after this
BH-9 Doc sections	27 Sep 18:00	Normalization and feature sections sent to Arushi

BH-1 prompt: data checks

Task BH-1. Implement `src/eda.py` (`run_checks`) using only `io_utils` readers. Print results and also write them to `docs/EDA.md` as short sections with tables.

Answer, with numbers:

- (a) Does any S2/S3 id appear in the matched list of 2+ Source 1 entities in train ground truth? Count them.
- (b) For every ground-truth pair, is the S1 country string identical to the matched record's country? Show a crosstab of mismatches, and list all distinct country strings per split and source (with counts), including stray variants like "USA"/"United States".
- (c) Singleton rate and the distribution of match counts per S1 (0,1,2,3,4,5+), overall and per country; also how many matches come from S2 vs S3.
- (d) How often an S1 has 2+ matches from the SAME source.
- (e) Row counts per split x source x country; share of train S2/S3 records that match nothing.
- (f) 25 random matched pairs S1<->S2 and 25 S1<->S3 printed side by side (name | address), per country, so we can see the noise patterns.
- (g) 40 random test France records from each source (inputs only), plus the 30 most frequent name tokens and address tokens for France.
- (h) Spearman correlation between the numeric part of matched S1 and S2/S3 ids. Report only.

Also report: empty-name and empty-address counts per source, name/address length percentiles, and the share of addresses containing 5- or 6-digit numbers.

Run it on the full data and show me the printed output.

Post (a), (b) and (c) in the team chat the moment you have them.

BH-2 prompt: normalization v0

Task BH-2 (v0, fast). In `src/normalize.py` implement `normalize_name`, `normalize_address` and `build_records`.

v0 only needs `name_norm`, `name_core`, `addr_norm`; fill the other INTERFACES columns with "" / [] / False.

- Accent strip: `unicodedata NFKD` then drop combining marks. Lowercase. Replace "&" and the words "and"/"et" with "and". Replace punctuation with spaces except keep digits and letters; collapse whitespace.
- `name_core` = `name_norm` minus legal-form tokens and phrases (`inc`, `incorporated`, `corp`, `corporation`, `co`, `company`, `llc`, `ltd`, `limited`, `pvt`, `private`, `plc`, `llp`, `lp`, `gmbh`, `sa`, `sas`, `sasu`, `sarl`, `eurl`, `sci`, `snc`, `"pvt ltd"`, `"private limited"`, `"p ltd"`) and minus leading "the". Never let `name_core` become empty: fall back to `name_norm`.
- `addr_norm` = same cleanup on the address.

`build_records`: `io_utils.read_all_records()` -> normalized columns -> save `config.cache_path("records.parquet")` with the exact INTERFACES column names and dtypes. Use vectorized pandas or a simple loop; it should finish in under 2 minutes. Print 10 random before/after examples per country.

BH-3 prompt: normalization v1

Task BH-3 (v1). Extend `src/normalize.py` to fill every column of `records.parquet` in `docs/INTERFACES.md`. The rules must not branch on country; map every variant to ONE short canonical token.

Names:

- `legal_form`: canonical class from the legal tokens (INC, CORP, LLC, LTD, PVT_LTD, CO, LLP, LP, PLC, GMBH, SA, SAS, SARL, EURL, SCI, SNC), "" if none. Handle dotted/joined forms: "pvt.ltd", "p. ltd", "(p) ltd", "l.l.c".
- `name_alts`: split on " dba ", "d/b/a", " t/a ", "trading as", " aka ", and on parentheses; each variant goes through the same core cleanup.
- Abbreviation map for names (both directions collapse to the short form): international->intl, manufacturing->mfg, services->svcs, service->svc, technologies/technology->tech, enterprises->ent, industries->ind, brothers->bros, associates->assoc, solutions->soln, company->co, and similar. Keep the map in one dict at the top of the file.

Addresses:

- Canonical tokens: street->st, saint->st, road->rd, avenue/av->ave, boulevard/bd->blvd, drive->dr, lane->ln, place->pl, suite/ste/#->ste, floor/flr->fl, apartment->apt, building/bldg->bldg, north/south/east/west->n/s/e/w, near/nr->nr, opposite/opp->opp, behind->bhnd, marg, nagar, colony, sector, phase, cross, main keep as-is; ordinals first->1st etc.; Roman numerals after phase/sector/stage -> digits.
- `addr_numbers`: all number-like tokens, with 12/3/456, 12-3-456, 12 3 456 unified to "12-3-456"; keep order.
- `postcode`: a standalone 5- or 6-digit token (last one wins); strip it from `addr_norm` into the column.
- `state`: map US state names <-> 2-letter codes and Indian state names <-> common abbreviations to one canonical form.
- `city`: best guess = the last comma-separated segment of the raw address that is not a state/postcode/country.
- `unit`: suite / floor / shop no. / flat no. value.
- `is_landmark` / `landmark`: true when the address uses nr / opp / bhnd / "next to"; landmark = the words after it up to the next comma.

Then print 30 random records per country as raw -> parsed fields, plus fill rates of each column per country and source. Show me the output.

BH-4 prompt: pair features v1

Task BH-4. Implement `src/features_pair.py`. Input: `cache/records.parquet` and `cache/candidates_{split}.parquet` (Siva). Output: `cache/feat_pair_{split}.parquet` with `s1_id`, `cand_id` and float32 features; NaN where a side is empty.

Name features (prefix `fn_`), computed on `name_norm`, `name_core`, and the best over `name_alts` variants:

- rapidfuzz: `fuzz.ratio`, `partial_ratio`, `token_sort_ratio`, `token_set_ratio`, `WRatio`, `JaroWinkler.normalized_similarity`, `Levenshtein.normalized_similarity` (scale to 0-1)
- char 3-gram Jaccard; token Jaccard; exact core match flag; length diff; token count diff
- TF-IDF cosine: `fit TfidfVectorizer(analyzer="char_wb", ngram_range=(2,4))` and a word-level one on ALL `name_core` values from train+test records together; cosine per pair via row-wise dot products of the normalized sparse rows
- legal form: both present and equal / both present and different / one missing (3 flags)

Address features (prefix `fa_`): the same fuzzy scores on `addr_norm`; TF-IDF cosine; postcode equal / conflict / missing; `addr_numbers` Jaccard; first-number equal / conflict; city equal; state equal; unit conflict; `is_landmark` on either side.

Performance: chunk the pairs, use `multiprocessing.Pool(config.N_JOBS)` with the `__main__` guard, and log progress. Target under 15 minutes on full train. Print `describe()` of every feature split by label (join `cache/labels.parquet`) so we can see which features separate matches.

BH-5 prompt: abbreviations, phonetics, French

Task BH-5. Three additions to `src/normalize.py`:

1) Mine abbreviation pairs from train ground truth: for each matched pair, align the `name_norm` tokens (difflib SequenceMatcher on token lists) and count replaced token pairs (a,b) where one is a prefix/subsequence of the other or they share the first letter. Keep pairs seen ≥ 5 times across ≥ 3 different entities. Save to `src/data/abbrev_mined.csv` and merge into the maps. Print the top 50.

2) `name_phon`: phonetic skeleton of `name_core`. Collapse repeated letters; `bh->b`, `dh->d`, `th->t`, `kh->k`, `gh->g`, `ph->f`, `sh->s`, `v->w`, `z->j`, `ee->i`, `oo->u`, `aa->a`, `ksh->x`; then a second variant with vowels removed except the first letter. Check that Shree/Shri/Sri, Laxmi/Lakshmi, Ganesh/Ganesha collapse together while unrelated names stay apart and show me.

3) French forms from docs/EDA.md (g): legal forms (SARL, SAS, SASU, SA, EURL, SCI, SNC, "societe", "ste" as a leading word, "ets", "cie"), street types (rue, bd, av, pl, ch/chemin, imp/impasse, all/allee, quai, rte/route, cours), "bis"/"ter" after house numbers, CEDEX removal, arrondissement forms ("75002 Paris", "Paris 2e"). Stopwords de, la, le, du, des, l, d. Still no country branching.

Re-run `build_records` and show 30 France test records raw -> parsed.

BH-6 prompt: pair features v2

Task BH-6. Add discriminator features to `src/features_pair.py` (prefix `fn_ / fa_`), aimed at look-alikes such as "Acme Robotics" vs "Acme Bakery" at the same address:

- IDF (from all records) of the rarest token present in only one of the two `name_core` values; $\text{sum IDF of shared tokens} / \text{sum IDF of all tokens}$
- numbers in names: both have numbers and they differ ("Store 12" vs "Store 14")
- acronym match: initials of one name equal a token of the other ("ibm" vs "international business machines")
- Monge-Elkan similarity with Jaro-Winkler as inner metric, both directions, max and mean
- first-token equal; last-token equal
- phonetic: ratio and `token_set_ratio` on `name_phon`
- landmark overlap: token Jaccard between a landmark phrase and the other side's full address
- best `fn_` scores over all `name_alts` combinations

Keep `v1` columns unchanged. Print the new features' `describe()` by label and the runtime.

BH-7 prompt: error-sheet loop (repeat after each model run)

Here is docs/errors/<run_id>_fp.csv and _fn.csv from Arushi (worst false positives and false negatives with raw fields and model probability). Group the errors by cause: normalization miss (show the raw and normalized strings), missing feature, genuinely ambiguous, or label noise. For the top 3 fixable causes, propose the smallest change to normalize.py or features_pair.py, implement it, and re-run on the affected rows to show before/after values.

BH-9 prompt: methodology sections

Write the "Normalization" and "Feature engineering" sections for our methodology document from the current code in src/normalize.py and src/features_pair.py and docs/EDA.md. Include: every rule family with 2 examples each, the full feature list grouped by family with one line each, fill rates, and what we learned from the data checks. Plain markdown, no marketing language, under 1,200 words.

Watch out for: a name_core that becomes empty after stripping ("The Company Ltd"); "st" meaning street vs saint (both map to st, on purpose); leaking labels into features (never read ground truth inside features_pair.py, only when printing diagnostics).

Siva: blocking and context features

You decide which S2/S3 records the model ever looks at for each S1, so your recall is the ceiling on our score. Target: at least 99% of ground-truth pairs found, with roughly 30–50 candidates per S1. Your candidates_{split}.parquet is also exactly what gets written to candidate_pairs.tsv, which the judges audit.

Files you own: src/blocking.py, src/features_context.py.

Task	Due (IST)	Done when
SI-1 Blocking v0	25 Sep 14:00	Name TF-IDF pass runs on train and test; candidates v0 saved
SI-2 Blocking report	25 Sep 15:00	Pair recall, oracle F0.5 and candidates/S1 printed per pass and per country
SI-3 All passes + pruning	25 Sep 19:00	Candidates v1 saved with all INTERFACES columns; pair recall $\geq$ 99% on train
SI-4 Context features	26 Sep 13:00	feat_ctx_{split}.parquet, one row per candidate, fc_* columns

Task	Due (IST)	Done when
SI-5 Stage-2 features	26 Sep 15:00	<code>feat_ctx2_{split}_{run_id}.parquet</code> built from out-of-fold predictions
SI-6 Pruning v2, miss study, France	26 Sep 21:00	Recall same or better at a smaller N; notes on missed pairs; France stats posted
SI-7 Freeze	27 Sep 14:00	Final test candidates saved; no more blocking changes
SI-8 Doc section + clean rerun	27 Sep 18:00	Section sent to Arushi; pipeline reproduced from raw data in a fresh environment

Until Bhanu's records v0 lands (12:00), build against a quick lowercase version of `read_all_records()` so you are not blocked.

SI-1 prompt: blocking v0

Task SI-1. Implement the first pass ("tfidf_name") in `src/blocking.py` and a `generate_candidates(split)` that saves `cache/candidates_{split}.parquet` with the INTERFACES columns (`blk_/bs_` columns for passes not built yet = False / NaN).

- Input: `cache/records.parquet (name_core)`. If it does not exist yet, fall back to lowercased `business_name` from `io_utils.read_all_records()`.
- For each split, for each country value present among that split's S1 records (loop over the data, never a hard-coded list), and for each target source (2 and 3) separately: fit `TfidfVectorizer(analyzer="char_wb", ngram_range=(3,5), min_df=1, sublinear_tf=True)` on the `name_core` of all records of that split (all sources, all countries), transform S1 and target records of that country, and take the top K=20 cosine neighbours per S1.
- Do the matrix product in chunks of 2,000 S1 rows (`sparse @ sparse.T`), then take top-K per row with `numpy argpartition` on the chunk's data. Never build a dense S1 x pool matrix.
- If a country has no records in the target source, fall back to the whole target pool of that split.
- When `config.SAMPLE_FRAC < 1`, keep only S1 ids where `io_utils.in_dev_sample(id)` is true; the S2/S3 pool always stays complete. Print runtime and candidates per S1 (mean, p50, p95) per country.

SI-2 prompt: blocking report

Task SI-2. Add a `report(split="train")` function to `src/blocking.py` (use `metrics.blocking_report` from Arushi if it exists; otherwise write a local helper with the same signature and swap later).

For train, using `io_utils.read_ground_truth()`:

- pair recall = found ground-truth pairs / all ground-truth pairs; per pass, for the union, per country, per target source
- entity-level recall ceiling = share of S1 whose true matches are ALL in the candidates
- oracle F0.5 = macro F0.5 if we predicted exactly the true matches that are inside the candidates
- candidates per S1 (mean, p50, p95) and total pairs
- marginal recall per pass = recall lost if that pass were removed

Print as a table and append a row to a small CSV `docs/blocking_log.csv` (date, passes, K/N, recall, oracle F0.5, mean candidates).

SI-3 prompt: all passes and pruning

Task SI-3. Add these passes to `src/blocking.py`, each returning `DataFrame[s1_id, cand_id, score]` and each run within country and per target source like `tfidf_name`:

- 1) `tfidf_full`: same TF-IDF on `name_core + " " + addr_norm`, `K=20`
- 2) `rare_token`: inverted index on `name_core` tokens; pair S1 and target records that share a token whose document frequency in that split is `<= 20` (skip tokens that are pure digits or `<= 2` chars); `score = max IDF` of the shared tokens; cap 30 per S1 by score
- 3) `addr_key`: exact match on (`postcode`, first entry of `addr_numbers`) when both are non-empty; also on (`city`, first number, first street token) as a second key; cap 30 per S1
- 4) `dense`: read `cache/emb/dense_neighbors_{split}.parquet` from Adithya if it exists, else skip with a warning
- 5) `reverse`: for every S2/S3 record, its top 5 S1 by the `tfidf_name` cosine (same country); add those (`s1`, `cand`) pairs

Union all passes into one row per (`s1_id`, `cand_id`) with `blk_<pass>` flags and `bs_<pass>` scores. `prune_score` = max over passes of the pass score converted to its within-pass percentile rank (so passes are comparable). Keep the top N per S1 by `prune_score`; N is a CLI arg, default 50; always keep pairs found by 3 or more passes. Add `prune_rank`.

Run the SI-2 report for N in [20, 30, 50, 80] and show recall vs mean candidates, so we can pick N.

SI-4 prompt: context features

Task SI-4. Implement `build_context_features(split)` in `src/features_context.py`. Output `cache/feat_ctx_{split}.parquet`: `s1_id`, `cand_id` and float32 `fc_*` columns, one row per candidate row.

- `fc_name_freq_s1 / fc_name_freq_cand`: how many records in that split share the exact `name_core` (chains)
- `fc_addr_freq_s1 / fc_addr_freq_cand`: how many distinct `name_core` values share the same `addr_norm`, or the same (postcode, first number) (malls, business parks)
- `fc_n_cands`: candidates for this S1; `fc_n_close`: candidates for this S1 with `bs_tfidf_name` ≥ 0.8
- `fc_rank`: `prune_rank`; `fc_gap_best`: best `prune_score` of this S1 minus this pair's
- `fc_rev_rank`: rank of this S1 among all S1s that have this `cand_id` as a candidate (by `prune_score`), and `fc_rev_n`: how many S1s list this `cand_id`
- `fc_n_passes` and each `blk_flag` as 0/1, each `bs_score` (NaN when missing)
- `fc_cand_source`: 2 or 3

No ground truth anywhere in this file. Print `describe()` by label as a diagnostic in `__main__` only.

SI-5 prompt: stage-2 features

Task SI-5. Implement `build_stage2_features(split, run_id)` in `src/features_context.py`. Input: `cache/preds/{run_id}_{split}.parquet` (on train these are out-of-fold probabilities; never use in-fold ones). Output `cache/feat_ctx2_{split}_{run_id}.parquet` with `fc2_*` columns:

- `fc2_rank_p`: rank of this pair's prob within the S1; `fc2_gap_p`: max prob of the S1 minus this prob; `fc2_second_p`: the S1's second-highest prob
- `fc2_rev_rank_p`: rank of this S1 among all S1s that list this `cand_id`, by prob; `fc2_mutual_best`: 1 if this pair is both the S1's best and the candidate's best S1
- `fc2_best_competitor_p`: highest prob of this `cand_id` with any OTHER S1
- `fc2_support`: for this candidate `c` of S1 `e`, max over `e`'s other candidates `a` from the OTHER source of $p(e,a) * \text{token_set_ratio}(\text{name_core}(a), \text{name_core}(c)) / 100$ (an S2 and an S3 record that both look like `e` and like each other support each other)
- `fc2_n_above_05`: number of the S1's candidates with prob > 0.5

Keep it vectorized with `groupby`; `fc2_support` can loop per S1 since lists are short.

SI-6 prompt: pruning v2, misses, France

Task SI-6. Three things:

- 1) Learned pruning: on train, fit LogisticRegression on [bs_* filled with -1, blk_* flags, n_passes] to predict the label (GroupKFold by s1_id for the evaluation), use its probability as prune_score, and re-run the recall-vs-N table. Keep it only if recall at the same N improves.
- 2) Misses: list the ground-truth pairs no pass finds (raw name and address of both sides, country, source). Group them by cause (name very different, address-only match, empty fields, cross-country, other) and print counts plus 10 examples per group.
- 3) France: for test, print candidates per S1 and the distribution of bs_tfidf_name for the best candidate per S1, by country (US, India, France). Flag if France looks shifted.

SI-8 prompt: methodology section

Write the "Candidate generation / blocking" section of our methodology document from src/blocking.py and docs/blocking_log.csv. Include each pass with its purpose and parameters, the pruning rule, a table of per-pass marginal recall, final pair recall and oracle F0.5 on train, mean candidates per S1, and the reduction ratio (1 - candidate pairs / all possible S1 x S2/S3 pairs in the same split). Plain markdown, under 900 words.

Watch out for: memory blow-ups from dense matrices (always chunk); duplicate (s1_id, cand_id) rows after the union; candidates that point at an ID from the other split. Every cand_id must exist in the same split's S2/S3 files.

Arushi: metric, match selection, submissions, packaging

You own the number everyone is chasing and the final step that turns probabilities into matches. The selection step alone can move our score by several points, because an empty-vs-non-empty mistake costs a full 1.0 on that entity. You also run every upload and build the final zip.

Files you own: src/metrics.py, src/folds.py, src/postprocess.py, src/evaluate.py (new), the writers in src/io_utils.py, scripts/, docs/experiments.csv, docs/errors/.

Task	Due (IST)	Done when
AR-1 Labels, folds, metric	25 Sep 13:00	<code>labels.parquet</code> and <code>folds.parquet</code> saved; metric tests pass, including the PDF example (0.714)
AR-2 Writers + baseline	25 Sep 16:00	Validator prints PASS; submission #1 uploaded and logged
AR-3 Log + leave-one-country-out scoring	25 Sep 20:00	<code>evaluate.py</code> prints CV and leave-one-country-out F0.5 for any predictions file
AR-4 Selection v1	25 Sep 22:00	Tuned threshold (+ exclusivity if BH-1 allows) beats the untuned 0.5 threshold on out-of-fold predictions
AR-5 Calibration + expected-F0.5 selection	26 Sep 17:00	Chosen method wins on both out-of-fold and leave-one-country-out scores
AR-6 Error sheets	After every model run	<code>docs/errors/<run_id>_fp.csv</code> and <code>_fn.csv</code> posted
AR-7 Submissions	Daily	Every upload has a validator PASS and a log row with the public score
AR-8 Final zip	27 Sep 21:00	<code>Chai-Square_submission.zip</code> rebuilt from a clean run; validator PASS
AR-9 Methodology doc	27 Sep 21:00	<code>Documentation_template.md</code> filled from everyone's sections

AR-1 prompt: labels, folds, metric

Task AR-1. Implement `src/metrics.py` and `src/folds.py`.

`metrics.py`:

- `f05_entity(pred, true)`: both empty -> 1.0; exactly one empty -> 0.0; else $TP = |\text{pred} \ \& \ \text{true}|$, $P = TP / |\text{pred}|$, $R = TP / |\text{true}|$, $F = 1.25 * P * R / (0.25 * P + R)$ (0 when $TP=0$).
- `macro_f05(pred, true)`: mean over every key of `true`; keys missing from `pred` count as empty predictions. Also return a per-entity Series when asked (`return_per_entity=True`).
- `blocking_report(candidates, truth)`: `pair_recall`, `entity_ceiling` (share of S1 whose true matches are all in candidates), `oracle_f05` (predict exactly true matches inside the candidates), `mean/p50/p95` candidates per S1.
- tests in `tests/test_metrics.py` (pytest): the problem-statement example {S2-00047,S2-00193,S3-00812} vs {S2-00047,S3-00812} = 0.714 (3 decimals), all empty cases, a perfect prediction = 1.0.

`folds.py` (`make_labels_and_folds`):

- `cache/labels.parquet`: `s1_id`, `cand_id`, `label=1` for every ground-truth pair (explode the matches list).
- `cache/folds.parquet`: every train S1 (singletons too) with `country`, `n_true`, `fold` (0..4) from `StratifiedKFold(5, shuffle=True, random_state=config.SEED)` stratified on `country + "_" + min(n_true, 3)`; plus `loco_fold` = integer code of the country (so a model can train on one country and predict the other). Print fold sizes and the singleton rate per fold.

AR-2 prompt: writers and baseline submission

Task AR-2. (1) Implement `io_utils.write_id_lists` following its docstring exactly: rows in the order of the test `source1` file, header

`source1_entity_id<TAB><list_col>`, comma-joined ids with no spaces, deduplicated in first-seen order, "" for none, UTF-8, "\n" line endings; assert every id starts with S2- or S3- and exists in the test S2/S3 files.

(2) Add `scripts/validate.py` that runs `utils/validate_submission.py` with our paths via subprocess and exits non-zero on failure. (3) Baseline in `scripts/baseline.py`: for each S1, TF-IDF `char_wb` (3,5) cosine on lowercased "name address" within the same country, the best S2 and the best S3 candidate; predict each one whose cosine $\geq t$. Tune `t` on train over a 0.30..0.95 grid with `metrics.macro_f05`, print the curve, then write `output/matching_results.tsv` and `output/candidate_pairs.tsv` (candidates = the top 10 per source per S1) for test with the best `t` and run the validator.

AR-3 prompt: evaluation and the experiment log

Task AR-3. Create `src/evaluate.py` with a CLI: `python -m src.evaluate --run-id <run_id> [--params params.json]`.

- Load `cache/preds/<run_id>_train.parquet` (out-of-fold probabilities), ground truth and folds.
- Apply `postprocess.select_matches` with the given params (or a default 0.5 threshold until AR-4 exists) and report macro F0.5 overall, per fold, per country, per n_true bucket (0,1,2,3+), and per source; also the singleton accuracy (share of true singletons predicted empty) and the non-singleton empty rate.
- If a leave-one-country-out predictions file `cache/preds/<run_id>_loco_train.parquet` exists (Adithya trains it with `train.py` using the `loco_fold` column), report its F0.5 per held-out country too.
- Append one row to `docs/experiments.csv` with the existing columns (fill what you can, leave `public_lb` empty).

AR-4 prompt: selection v1

Task AR-4. Implement `postprocess.select_matches(preds, params)` and `tune(oof_preds, truth)`.

- `params` keys: `threshold` (float), `per_source` (bool; then `threshold_s2` and `threshold_s3`), `exclusive` (bool).
- `exclusive`: for every `cand_id` keep only the row with the highest prob across S1s before thresholding (ties -> keep both out, i.e. drop).
- `selection`: predict every candidate with `prob >= threshold` (per source if set).
- `tune`: grid search thresholds 0.05..0.95 step 0.01 on out-of-fold predictions maximizing `metrics.macro_f05`; try `exclusive` on/off and `per_source` on/off; return the best params and print the F0.5 curve so we see how flat the optimum is. Save to `models/<run_id>/postprocess.json`.

AR-5 prompt: calibration and expected-F0.5 selection

Task AR-5. Extend `src/postprocess.py`:

- 1) Calibration: fit `sklearn IsotonicRegression(out_of_bounds="clip")` on out-of-fold prob vs label, cross-fitted by fold so the calibrated train probs stay out-of-fold; for test fit on all train. Report Brier score before/after.
- 2) Expected-F0.5 set selection per S1 (`method="expected_f"`): sort that S1's candidates by calibrated p (after exclusivity). For each k in $0..n$ ($n \leq 50$), the prediction is the top k . Treat labels as independent Bernoulli(p_i). Let A = number of true among the top k (Poisson-binomial over $p_1..p_k$), B = number of true among the rest (Poisson-binomial over $p_{k+1}..p_n$); compute both distributions with the standard $O(n^2)$ DP. Then $E[F(k)] = \sum_{a,b} P(A=a)P(B=b) \cdot f(a,b,k)$ where $f = 1.25a / (0.25(a+b)+k)$ for $k > 0$ (0 when $a=0$), and for $k=0$, $E[F(0)] = P(A+B=0) = \prod (1-p_i)$. Pick the k with the highest expected value. Optional param `miss_rate m`: add m expected true matches outside the candidates by treating B as $B + \text{Poisson}(m)$ approximated by one extra Bernoulli(m).
- 3) Optional gate: a LogisticRegression on per-S1 features (`max p`, `second p`, `count p > 0.5`, `fc_n_close` if present) predicting "has any match", cross-fitted; if gate prob $< g$, predict empty.
- 4) Compare threshold vs `expected_f` vs `expected_f+gate` on out-of-fold AND leave-one-country-out predictions; print a table, pick the winner, save params.

Unit test: a single candidate with $p=0.6$ selects it; $p=0.4$ selects nothing; two candidates $[0.99, 0.7]$ selects only the first (the second needs about 0.73).

AR-6 prompt: error sheets

Task AR-6. Add `scripts/error_sheets.py --run-id <run_id>`: using the out-of-fold predictions and the chosen postprocess params, write `docs/errors/<run_id>_fp.csv` (100 predicted pairs that are wrong, highest prob first) and `<run_id>_fn.csv` (100 true pairs we missed, split into "not in candidates" and "in candidates but not selected", lowest prob first). Columns: `s1_id`, `cand_id`, `prob`, `rank within S1`, `country`, `source`, raw name and address of both sides, `name_core` and `addr_norm` of both sides, and the entity's true match count. Also print the entity-level loss breakdown: points lost on singletons we matched, on non-singletons we left empty, and on partial sets.

AR-7: submission checklist (no prompt)

1. Adithya names the run and the postprocess params to submit.
2. Build outputs with `run_pipeline.py` or the run's predict + postprocess, then run `python scripts/validate.py`.

3. Add the row in `docs/experiments.csv` (what changed, CV, leave-one-country-out) before uploading.
4. Upload `output/matching_results.tsv` from your laptop only; record the public score in the same row; tag the commit: `git tag sub-<nn>` and push the tag.
5. Never spend the last slot of a day without asking the team at the evening sync.

AR-8 prompt: final zip

Task AR-8. Implement `scripts/make_submission_zip.py`: build Chai-Square_submission.zip with `output/matching_results.tsv`, `output/candidate_pairs.tsv`, `code/business_entity_resolution/` (a copy of `src/`, `scripts/`, `README.md`, `requirements.txt`, `docs/INTERFACES.md`; exclude `dataset/`, `cache/`, `models/`, `output/`, `__pycache__`, `.git`) and `Documentation_template.md` at the root. Before zipping: run the validator, check every matched id is inside `candidate_pairs` for the same S1, and print the zip's file list and size. Also regenerate `requirements.txt` pins from `'pip freeze'` for exactly the packages already listed.

AR-9 prompt: methodology document

Fill `docs/Documentation_template.md` using the sections sent by Bhanu (normalization, features), Siva (blocking), Adithya (model), plus my own: validation design (folds, leave-one-country-out as the proxy for unseen France), the metric, calibration and the expected-F0.5 selection with its formula, and the submission history table from `docs/experiments.csv`. Keep the template's headings, make the numbers consistent across sections, and cut repetition. Plain markdown.

Watch out for: `nan` strings in the output (write `""`); training-set IDs leaking into test outputs (IDs overlap between splits); tuning thresholds on in-fold predictions (always out-of-fold); spending a submission without a log row.

Reporting back

At every sync, each person says three things in under a minute: the number they moved, what they handed off, and what is blocking them.

Who	Number to report
Bhanu	Fill rate of parsed fields; which new features rank high in Adithya's importance list

Who	Number to report
Siva	Pair recall, oracle F0.5, mean candidates per S1 (train)
Arushi	Best out-of-fold F0.5, leave-one-country-out F0.5, public score of the last upload

Pull requests: one task per PR, title `BH-3: normalization v1` style. The description has the command you ran and the printed numbers. Tag Adithya as reviewer. Merge at least twice a day so nobody drifts.

Escalate in the team chat immediately when:

- a handoff will miss its due time by more than an hour (say the new time, and ship a v0 if you can);
- you need a change to `docs/INTERFACES.md`, `config.py` or someone else's file;
- a full-data run takes more than 30 minutes or runs out of memory;
- the AI assistant suggests external data, a web lookup, a GPL library or hard-coded country lists (the answer is no);
- a number looks too good (for example, F0.5 above 0.98): it is usually a leak from in-fold predictions or labels inside features.